DATABASE PROJECT REPORT

Library Management System

Review 3 - Final Project Report

Submitted by

L Srinath - 24BDS0265
Sugash K - 24BCE2565

Course Code: BCSE302P
Course Title: Database Systems Lab

Source code: **Download the companion source archive**. It contains the full application and project files.

Prepared: 23 October 2026

Abstract

Academic libraries need a dependable way to manage bibliographic records, member accounts, loans, returns, reservations and overdue follow-up. Paper registers and disconnected spreadsheets make it difficult to keep stock counts consistent, trace circulation history, locate records quickly and prepare operational summaries. This project designs and implements a Library Management System using MongoDB, with a Node.js and Express application providing a responsive staff interface and protected REST API. The database contains six collections: authors, books, members, loans, reservations and users. A deterministic seed creates 137 sample documents. The model uses references for entities with independent lifecycles and embeds copy totals in book documents so inventory values are read together. Mongoose schemas and MongoDB JSON Schema validators enforce required fields and value domains; unique, text, compound and partial indexes support catalog, member, loan and reservation access. CRUD routes manage books and members. Issue, return and renewal workflows use multi-document transactions to coordinate loan status, inventory and reservation state. Eligible loans can be renewed twice, but not when overdue or reserved. Six aggregation reports summarize category inventory, popular books, overdue loans and fines, member activity, monthly circulation and reservation queues. The implementation is packaged with Docker Compose, source code, a seed script, unit tests, API smoke checks, a user manual and review documentation. This report follows the database life cycle from requirements through conceptual, logical and physical design, implementation and verification. The sample data and screenshots demonstrate the intended classroom workflows. No controlled performance benchmark or production deployment was performed. The Docker replica set is single-node for transaction support, not a high-availability configuration. Within that scope, the project demonstrates a reproducible document-database solution and identifies practical next steps for production security, backup, recovery and scale.

Table of Contents

Chapter 1: Introduction . . . 6

1.1 Background of the Study . . . 6

1.2 Motivation . . . 6

1.3 Problem Statement . . . 6

1.4 Aim and Objectives . . . 6

1.5 Scope of the Project . . . 6

1.6 Limitations . . . 6

1.7 Methodology Overview . . . 7

Chapter 2: Literature Review . . . 7

2.1 Overview of the Application Domain and Its Data Needs . . . 7

2.2 Review of Data Models . . . 7

2.3 Review of Existing Systems and Related Work . . . 7

2.4 Comparison of DBMS Platforms and Tools . . . 8

2.5 Standards and Best Practices . . . 8

2.6 Gap Identified . . . 8

2.7 How This Project Addresses the Gap . . . 8

2.8 Summary of the Chapter . . . 8

Chapter 3: System Analysis and Requirements . . . 9

3.1 Existing System and Its Drawbacks . . . 9

3.2 Proposed System and Its Advantages . . . 9

3.3 Feasibility Study . . . 9

3.3.1 Technical Feasibility . . . 9

3.3.2 Economic Feasibility . . . 9

3.3.3 Operational Feasibility . . . 9

3.4 Users, Roles and Access Needs . . . 9

3.5 Requirements Specification . . . 10

3.6 Business Rules and Constraints . . . 11

Chapter 4: Database Design . . . 12

4.1 Design Approach and Principles . . . 12

4.2 System Architecture Diagram . . . 12

4.3 Data Flow Diagram . . . 12

4.4 Conceptual Design: ER / EER Diagram . . . 13

4.4.1 Entities and Attributes . . . 13

4.4.2 Relationships, Cardinality and Participation . . . 13

4.4.3 Keys . . . 13

4.4.4 Generalization and Specialization . . . 14

4.5 Logical Design . . . 14

4.6 Normalization . . . 14

4.6.1 Functional Dependencies . . . 14

4.6.2 Normal Forms and Mapping . . . 15
4.6.3 Justified Denormalization . . . 15
4.7 Physical Design . . . 15
4.7.1 Data Types and Storage . . . 15
4.7.2 Indexing Strategy . . . 15
4.7.3 Partitioning . . . 15
4.8 Integrity Constraints . . . 15

Chapter 5: Implementation . . . 16
5.1 Development Environment and Setup . . . 16
5.2 DBMS, Tools and Libraries Used . . . 16
5.3 Collection and Validation Creation . . . 16
5.4 Initial Data Loading . . . 16
5.5 Views and Indexes Created . . . 16
5.6 CRUD, Transaction and Aggregation Modules . . . 16
5.7 Application Connectivity . . . 16
5.8 Version Control and Project Repository . . . 17
5.9 Challenges Faced and Solutions Adopted . . . 17

Chapter 6: Query Design and Performance Tuning . . . 18
6.1 Representative Queries . . . 18
6.2 Performance Tuning Strategy . . . 18
6.3 Query Plan and Benchmark Protocol . . . 18

Chapter 7: Results and Discussion . . . 20
7.1 Output Screenshots and Sample Reports . . . 20
7.2 Evaluation Against Objectives . . . 22
7.3 Comparison with the Existing System . . . 23
7.4 Discussion of Findings . . . 23
7.5 Limitations Observed . . . 23

Chapter 8: Conclusion and Future Work . . . 24
8.1 Summary of the Work . . . 24
8.2 Key Contributions . . . 24
8.3 Future Enhancements . . . 24
8.4 Concluding Remarks . . . 24

Part C: References . . . 25

Appendix A: Complete Database Schema and Index Definitions . . . 26
author.js . . . 26
book.js . . . 26
member.js . . . 26
loan.js . . . 26
reservation.js . . . 27
user.js . . . 27

Appendix B: Sample Data and DML Script . . . 28

B.1 Full seed implementation . . . 28

Appendix C: Stored Procedures, Functions and Transactions . . . 30

C.1 Circulation routes . . . 30

C.2 Date and fine functions . . . 32

Appendix D: Complete Query Set with Outputs . . . 33

D.1 Analytics route source . . . 33

D.2 API query examples . . . 34

D.3 Seed-state output evidence . . . 34

Appendix E: Full-Size ER Diagram and Logical Schema . . . 35

Appendix F: Application Screenshots . . . 36

Appendix G: User Manual . . . 39

G.1 Start the application . . . 39

G.2 Demonstration accounts . . . 39

G.3 Core workflows . . . 39

G.4 Manual backup and restore . . . 39

G.5 Troubleshooting . . . 39

G.6 Verification commands . . . 39

Chapter 1: Introduction

1.1 Background of the Study

A library manages connected resources: bibliographic records, copies, authors, members, loans, reservations and staff accounts. Circulation is stateful: issuing a book changes both loan history and available stock; a return can change fines and the reservation queue. A suitable database system therefore needs stable identifiers, validation, search and consistent multi-step updates.

1.2 Motivation

The project replaces fragmented classroom demonstrations of catalog and circulation work with one reproducible application. It demonstrates how a database model supports real operations, rather than storing isolated records. Reports also make NoSQL capabilities observable to an evaluator.

1.3 Problem Statement

Paper and spreadsheet workflows can duplicate entries, lose links between issue and return, and leave availability inconsistent after partial updates. They also require manual grouping to identify overdue items, popular books and reservation order. This project addresses those problems for a small academic-library demonstration using one database and role-protected operations.

1.4 Aim and Objectives

ID	Objective	Evidence
O1	Design a document model for catalog, authors, members, circulation, reservations and staff.	ER model, collection schemas, validators and seed counts
O2	Load repeatable sample data above the minimum record requirement.	137 seeded documents across six collections
O3	Implement CRUD with application rules.	Books and members routes plus circulation interface
O4	Use indexes and aggregation for retrieval and management reports.	Model index definitions and six live report pipelines
O5	Maintain consistency for issue/return operations spanning documents.	Transaction routes and workflow evidence
O6	Package the app and instructions for repeatable demonstration.	Docker Compose, source archive, manual and demo guide

1.5 Scope of the Project

Included: staff login and roles; books and members CRUD; issue, return, reservation and policy-controlled renewal workflows; overdue fines; six aggregation reports; JSON Schema validation; indexes; responsive staff UI; Docker-based classroom setup. Excluded: patron self-service, barcode/RFID, email notices, acquisitions, inter-library circulation, multi-branch policy, live catalog import and production service-level guarantees.

1.6 Limitations

The demonstration data is synthetic and small (137 documents). Compose uses one MongoDB replica-set member for transaction support; it does not provide node redundancy or high availability. No controlled performance benchmark, production backup schedule, disaster-recovery exercise, TLS deployment or live institutional test is claimed. References between collections are application-level rather than MongoDB foreign-key constraints.

1.7 Methodology Overview

The database life cycle is followed from requirements through conceptual, logical and physical design, implementation, testing and maintenance planning.

Stage	Work	Artifact
Requirements	Identify users, data, circulation rules and reports	Functional and non-functional requirements
Conceptual	Identify entities and relationships	ER/EER-style diagram
Logical	Map entities to collections, references and embedded values	Collection and field schema
Physical	Select BSON types, validators, indexes and replica-set setup	Mongoose models, seed, Compose
Implementation	Build API, UI, seed and pipelines	Node.js/Express application
Testing	Provide unit/API smoke checks and browser evidence	test/ and docs/evidence
Maintenance	Document reset, backup, password rotation and scale steps	Manual and future-work plan

Chapter 2: Literature Review

2.1 Overview of the Application Domain and Its Data Needs

The domain includes descriptive metadata, copy-level availability, member eligibility, circulation events, ordered holds and staff privileges. Some values are read together (book copy totals); other records have independent histories (loans and reservations). The workload combines indexed lookup, CRUD, reference resolution and grouped reports. MongoDB recommends designing around access patterns and selecting embedding or references according to read/update behavior [1]-[3].

2.2 Review of Data Models

Model	Typical fit	Strength	Fit for this project
Relational	Structured entities and joins	SQL, normalized relations and constraints	Strong alternative; entities could be separate tables
Object-relational	Relational data with richer types	SQL plus user-defined/complex types	Possible; no clear benefit for this JSON-shaped app
NoSQL document	Nested/evolving records	Documents, embedding/references, aggregation	Chosen; application objects and copy subdocument align
Key-value	Key lookup, cache, counters	Simple key operations	Not enough for primary multi-field reporting
Wide-column	Large distributed query-specific workloads	Partitioned, query-driven storage	Cassandra exceeds project needs [11]
Graph	Relationship path queries	First-class nodes and edges [12]	Possible future recommendation feature
NewSQL	Distributed SQL transactions	Relational semantics and scale	No distribution requirement in current scope

Why MongoDB was selected

MongoDB combines document storage, aggregation, schema validation, indexes and transactions [4]-[7]. PostgreSQL is a strong relational alternative with JSON/JSONB support [10], [17]; selection here is based on fit with the Node/Mongoose application and learning objectives, not a performance benchmark. Redis offers native data structures [13] and fits cache/queue use. Cassandra is query-driven [11]. Neo4j makes graph traversal central [12], whereas this workload is CRUD and report focused.

2.3 Review of Existing Systems and Related Work

Koha is an established open-source integrated library system. Its manual documents catalog and circulation workflows, patron permissions, check-outs, returns and renewals [14]. It has a broader operational scope than this coursework project. This application is a focused database-design implementation, not a replacement for Koha. Paper registers and disconnected spreadsheets have low setup cost but rely on manual reconciliation and reporting. No controlled product benchmark or user study was conducted.

2.4 Comparison of DBMS Platforms and Tools

Platform	Model/role	Relevant capabilities	Decision
MongoDB 7	Document DBMS	BSON, validators, indexes, aggregation, transactions	Selected
PostgreSQL 18	Relational + JSON	SQL constraints, JSON/JSONB [10], [17]	Strong alternative
Redis	In-memory data structures	Strings, hashes, sets and streams [13]	Not primary store
Apache Cassandra	Wide-column	Query-first distributed modeling [11]	Scale exceeds need
Neo4j	Property graph	Typed edges and traversal [12]	Paths not central
Node/Express + Mongoose	App/API + ODM	Routes, validation, sessions and indexes	Selected
Docker Compose	Local orchestration	Repeatable application/database setup	Selected

2.5 Standards and Best Practices

The system uses JSON-shaped documents stored as BSON and REST-style HTTP routes. MongoDB schema validation can reject writes violating configured rules [4]. Indexes accelerate suitable reads but add write/storage cost [5]. Aggregation stages support filtering, grouping and computation [6]; transactions coordinate writes spanning documents [7]. Passwords are hashed; the classroom seed uses bcrypt cost 12, while OWASP recommends Argon2id for new systems when available [15]. Express guidance recommends input validation, security headers, secure cookies and dependency hygiene [16].

2.6 Gap Identified

A static schema does not demonstrate that stock, loans and reservation order remain consistent through real operations, or that useful reports can be derived from stored records.

2.7 How This Project Addresses the Gap

The project connects its model to a working interface and API, loads 137 documents, presents six reports, uses indexes and validates coordinated circulation changes with transactions. Source files and saved screenshots provide inspectable evidence.

2.8 Summary of the Chapter

Relational, document, key-value, wide-column, graph and NewSQL models have different workload strengths. MongoDB was selected for this JSON-oriented application and its CRUD/report access pattern. The next chapter defines users and requirements.

Chapter 3: System Analysis and Requirements

3.1 Existing System and Its Drawbacks

The baseline problem is a manual or spreadsheet workflow where catalog, borrower, due-date and hold information may be maintained separately. Reconciliation, search, audit history and reporting then rely on manual work. This is the project framing, not a measured study of a named institution.

3.2 Proposed System and Its Advantages

One MongoDB-backed application supplies controlled CRUD, role checks, circulation transactions, search, reservations and reports. The UI gives staff one workspace; the API enforces business rules; the database stores separate circulation history and book counters. A deterministic seed supports repeatable evaluation.

3.3 Feasibility Study

Dimension	Assessment
Technical	Feasible with Node.js, Express, MongoDB, Mongoose and Compose. Transactions require replica-set mode; the provided topology has one member.
Economic	Open-source dependencies and local execution avoid service costs for classroom work. Production hosting/support costs are not estimated.
Operational	Staff workflows are suitable for demonstration. Real deployment needs policy setup, training, migration, privacy review and support.

3.3.1 Technical Feasibility

The source and lockfile are included; Docker Compose encapsulates the app and database. Docker Desktop must be running. The previously reported manually paused engine is a local environment issue.

3.3.2 Economic Feasibility

Open-source tools and local execution avoid a database license or cloud subscription for the classroom prototype. No production total-cost estimate is claimed.

3.3.3 Operational Feasibility

Two staff roles and a focused interface support the demonstration. The user manual and timed demo guide support onboarding; no formal usability study was conducted.

3.4 Users, Roles and Access Needs

Role	Read	Write	Restricted actions
Administrator	All data/reports	Books, members and circulation	Only role with delete permissions
Librarian	Operational data/reports	Create/update; issue/return/reserve/renew	Cannot call admin-only delete
Unauthenticated	Login and health	None	Protected API returns 401

3.5 Requirements Specification

3.5.1 Functional Requirements

ID	Requirement	Implementation
FR-01	Staff login/logout and role checks	JWT cookie and auth middleware
FR-02	Books CRUD/search	Books routes and catalog UI
FR-03	Members CRUD/search	Member routes and UI
FR-04	Issue available book to active member	Transaction route
FR-04a	Renew eligible loan within policy	Transaction; max two, on/before due date, no active hold
FR-05	Return loan and calculate fine	Transaction; Rs.10/day
FR-06	Reserve unavailable title and order queue	Reservation routes/report
FR-07	Six reports and overdue CSV	Analytics routes/UI
FR-08	Repeatable sample load	Seed script; 137 records

3.5.2 Data Requirements

Entity	Source	Count	Growth assumption
Authors	Seed	20	Low
Books	Seed + staff	40	Hundreds for course demo
Members	Seed + registration	30	Small class/department
Loans	Seed + circulation	30	Highest growth; retain event history
Reservations	Seed + staff	15	Grows with demand
Users	Seeded staff	2	Low, controlled onboarding

Growth values are classroom assumptions, not a measured institutional forecast.

3.5.3 Non-Functional Requirements

Quality	Requirement and status
Performance	Bounded list queries and indexes; no latency benchmark measured.
Concurrency	Issue/return/renewal sessions and transactions; one database node.
Availability	Local demonstration; single-node replica set has no failover.
Security	bcrypt hash; 8-hour JWT; HTTP-only/same-site cookie; roles; Helmet; 500 requests/15 min. Local HTTP is not production TLS.
Backup/recovery	No automated backup; mongodump/mongorestore procedure documented; recovery must be tested [9].
Usability/maintenance	Responsive UI and separated source modules; no formal usability study.

3.5.4 Hardware Requirements

Estimated local setup: 64-bit laptop, 8 GB RAM, 2 CPU cores, 5 GB free disk and modern browser. These are practical estimates, not benchmark results.

3.5.5 Software Requirements

Docker Desktop/Compose, browser and terminal. Local alternative: Node.js 20+, npm, MongoDB 7 in replica-set mode; Compass optional. Libraries include Express 5, Mongoose 8, Zod 4, bcryptjs, jsonwebtoken, Helmet, compression, cookie-parser, express-rate-limit and pino-http.

3.6 Business Rules and Constraints

Rule	Enforcement
IDs use A###/B###/M### patterns	Zod and Mongoose
ISBN/email unique	Unique indexes
Non-negative counters; available <= total	Model/request checks; cross-field at application layer
Only active members borrow/reserve	Circulation route
No unavailable issue or duplicate active loan	Transaction and partial unique index
Return only active loan; stock max total	Transactional route predicates
Fine Rs.10/day and never negative	calculateFine helper and unit cases
Renew at most twice, through due date, only without active holds	Transactional route and unit tests
Reserve only when unavailable; oldest first	Route and queue sorting/index
Block member delete while active loan	Admin route

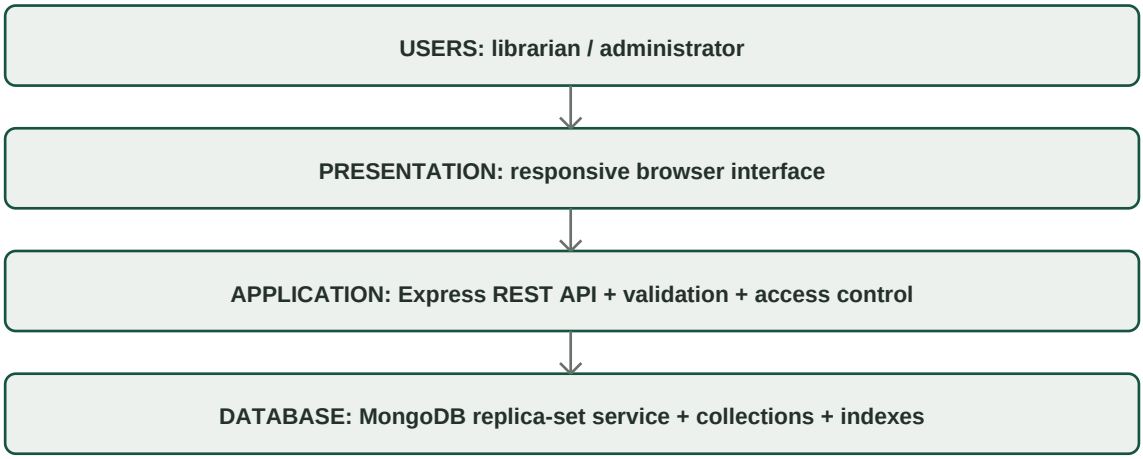
Chapter 4: Database Design

4.1 Design Approach and Principles

The model follows access patterns and separates entities whose histories grow independently. Author records are reused by ID; loans and reservations remain separate; a compact copy counter is embedded in books for atomic inventory changes. This balances references with a narrow, justified embedded subdocument [1]-[3].

4.2 System Architecture Diagram

The browser calls the Express API. Authentication and validation middleware protect routes. Mongoose maps application objects to MongoDB collections. Compose initializes a local single-node replica set. Backup/restore is a manual procedure, not an automated service.



Manual backup/restore is documented; no automated backup or redundant database node is configured.

Figure 1. Layered system architecture and classroom deployment boundary.

4.3 Data Flow Diagram

Issue flow reads a member and book, validates eligibility and availability, creates a loan and decrements stock in one transaction. Return flow closes the loan, calculates the fine, restores a copy and fulfils the oldest hold in one transaction.

Actor	Process	Stores touched	Output
Librarian	Issue book	members, books, loans, users	Loan confirmation and lower availability
Librarian	Return book	loans, books, reservations	Return/fine and possible fulfilled hold
Staff user	Request report	books, loans, members, reservations	Aggregated results

4.4 Conceptual Design: ER / EER Diagram

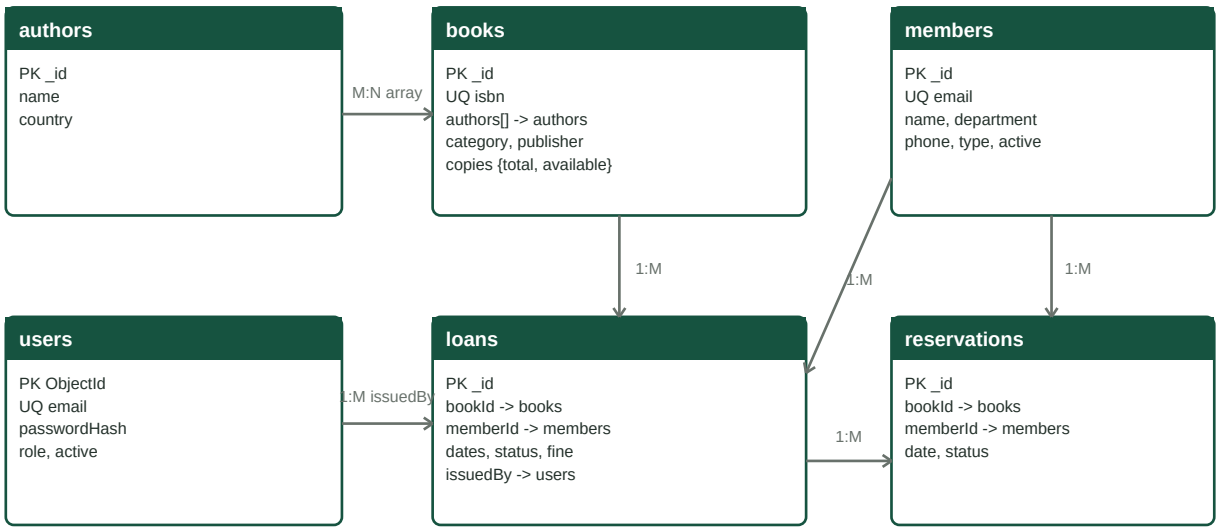


Figure 2. Entity relationships. Arrows show application references; book-author is represented as author IDs inside the book document.

4.4.1 Entities and Attributes

Entity	Identifier	Attributes
Author	A###	name, country
Book	B###	isbn, title, authors[], category, publisher, edition, year, copies
Member	M###	name, department, email, phone, membershipType, active
Loan	L###	bookId, memberId, issueDate, dueDate, returnDate, status, fine, issuedBy
Reservation	R###	bookId, memberId, reservationDate, status
User	ObjectId	name, email, passwordHash, role, active

4.4.2 Relationships, Cardinality and Participation

Relationship	Cardinality	Representation
Author writes Book	Many-to-many	Book stores array of author IDs; no bridge collection
Book has Loan	1:M over time	Loan references one book; book can have many historical loan records
Member borrows Loan	1:M over time	Loan references one member; member can have many loans
Book has Reservation	1:M	Reservation references a book; several members may queue
Member places Reservation	1:M	One member may hold several reservations; active duplicate pair blocked
User issues Loan	1:M	Loan stores staff user ID

4.4.3 Keys

Every collection has `_id` as its primary identifier. Unique secondary keys are books.isbn, members.email and users.email. Loan/reservation fields store references to related IDs. MongoDB does not enforce foreign keys; route checks and controlled writes preserve links. Member deletion is blocked while active loans exist. An operational version should include an orphan-reference reconciliation task.

4.4.4 Generalization and Specialization

No EER inheritance hierarchy is used. Student and Faculty share the members schema through membershipType; administrator and librarian accounts share users with a role enum.

4.5 Logical Design

Field	BSON/logical type	Size/domain	Constraint	Description
authors._id	String	A + 3 digits	PK, required, pattern	Author key
authors.name/country	String	Required	trimmed	Author description
books._id	String	B + 3 digits	PK, required	Catalog key
books.isbn	String	10-20 chars at API	Unique, required	ISBN
books.title	String	2-180 chars at API	Required, text-indexed	Title
books.authors	Array	A### refs	At least 1 at API	Author IDs
books.category/publisher	String	category <=60; publisher <=100	Required	Catalog metadata
books.edition/year	Number	edition 1-30; year 1900-2100	Range checked	Edition/publication
books.copies	Embedded object	total, available >=0	available <= total at app layer	Inventory counts
members._id	String	M + 3 digits	PK, required	Member key
members.name/department	String	2-100 chars at API	Required	Member details
members.email	String	Email	Unique, lowercase	Member identity
members.phone	String	10 digits at API	Required	Contact
members.membershipType	String	Student | Faculty	Enum	Member category
members.active	Boolean	true/false	Default true	Eligibility
loans._id	String	L + >=3 digits	PK, required	Circulation key
loans.bookId/memberId	String	B### / M###	Required refs	Borrowed entities
loans.issue/due/returnDate	Date	UTC date	Issue/due required	Circulation dates
loans.status	String	Issued | Returned	Enum	Loan state
loans.fine	Number	>=0	Default 0	Fine in rupees
loans.renewals	Number	0-2	Default 0; max 2	Renewal count
loans.issuedBy	ObjectId	users._id	Required ref	Staff actor
reservations._id	String	R + >=3 digits	PK, required	Hold key
reservations.bookId/memberId	String	B### / M###	Required refs	Held title/member
reservations.reservationDate	Date	UTC date	Required	Queue time
reservations.status	String	Active/Fulfilled/Cancelled	Enum	Hold state
users._id	ObjectId	Mongo generated	PK	Staff key
users.email	String	Email	Unique, lowercase	Login identity
users.passwordHash	String	bcrypt hash	Required, excluded by default	Credential verifier
users.role/active	String/Boolean	admin/librarian	Enum/default true	Access state

4.6 Normalization

Conceptually, authors, books, members, loans, reservations and users are separate entities. This avoids repeating author or member details in every circulation event. MongoDB references keep independent records addressable. The only intentional embedded group is copies within a book, because both counters are read and updated together.

4.6.1 Functional Dependencies

Entity	Functional dependency examples
Author	authorId -> name, country
Book	bookId -> ISBN, title, category, publisher, edition, year, copies
Member	memberId -> details; email -> memberId
Loan	loanId -> bookId, memberId, dates, status, fine, issuedBy
Reservation	reservationId -> bookId, memberId, date, status
User	userId -> details; email -> userId

4.6.2 Normal Forms and Mapping

Each conceptual entity's non-key attributes depend on its identifier in the logical model (1NF/2NF/3NF rationale). In MongoDB, the authors array is an application representation of a many-to-many relationship, not a relational bridge table. Loan details are not copied into members or books, preserving circulation history independently.

4.6.3 Justified Denormalization

Only books.copies {total, available} is embedded as a compact subdocument. Issue and return operations update stock on the same book record, avoiding a separate inventory collection. Author/member data remain referenced and report pipelines resolve them with `$lookup`. The trade-off is that stock counts remain a business invariant enforced by application logic.

4.7 Physical Design

Aspect	Decision
Storage	MongoDB 7 container; BSON; six collections; named Docker volume `lms_mongo_data`
Indexes	Unique ISBN/email; text book search; category+availability; member+status+due; book+status; partial unique active loan; reservation queue and partial unique active reservation; author name; member name+department
Partition/sharding	Not used; 137-doc classroom dataset, no scale test
Replication	One replica-set member for transaction support; not high availability [8]
Backup	No automation. Manual dump/restore; separate secured storage and restore rehearsal recommended [9]

4.7.1 Data Types and Storage

IDs use visible strings except MongoDB ObjectIds for users. Dates use BSON Date; active flags use Boolean; amounts/counts use Number; labels/statuses use String. The book stock object is embedded.

4.7.2 Indexing Strategy

Indexes target unique lookups, catalog search, category browse, member overdue queries and FIFO reservation lookup. Indexes reduce scanned records when a query can use them but increase storage and write cost; actual query plans should be reviewed as data and workload grow [5].

4.7.3 Partitioning

Not used. The current scale and course scope do not justify sharding or partitioning.

4.8 Integrity Constraints

Constraint	Implementation
Primary key	MongoDB `_id` in each collection
Unique	ISBN, member email, user email, partial active-loan/active-reservation pair
Required/type/enum	Mongoose, Zod request schemas and MongoDB JSON Schema validators
Foreign key	No MongoDB FK; `ref` IDs and application checks
Check	Non-negative values in schemas/validators; cross-field rules in route logic
Default	Mongoose defaults for active, role, edition, returnDate and fine
Referential action	No cascade; member deletion blocked with active loan; book deletion controlled

Chapter 5: Implementation

5.1 Development Environment and Setup

The project is a private npm ES-module application. Compose builds the app, starts MongoDB 7, initializes a single-node replica set and starts the web service. The seed command drops and recreates the `library_management` database, so it is a destructive reset and must only be used for demo data.

```
docker compose up -d --build
docker compose exec app npm run seed
docker compose ps
# Browse to http://localhost:3000
```

5.2 DBMS, Tools and Libraries Used

Component	Role/version
MongoDB 7	Document store, indexes, validation, aggregation, transactions
Node.js >=20	Application runtime
Express 5	REST routes and middleware
Mongoose 8	Schemas, references, indexes and sessions
Zod 4	Request validation
bcryptjs / jsonwebtoken	Password hashing / signed token
Helmet / express-rate-limit	Security headers / request throttling
Docker Compose	Local orchestration
MongoDB Compass	Optional visual inspection

5.3 Collection and Validation Creation

Mongoose models define field rules and indexes. The seed script inserts records, synchronizes declared indexes, then applies strict collection validators using `collMod`. MongoDB validators enforce core structure for direct database writes [4].

5.4 Initial Data Loading

The deterministic seed creates 2 users, 20 authors, 40 books, 30 members, 30 loans and 15 reservations (137 total). Data include active and returned loans, unavailable titles and both active/cancelled reservations. Records are synthetic, not real borrower information.

5.5 Views and Indexes Created

No server-side MongoDB `createView` view is created. Dashboard and report endpoints provide derived read-only presentation views. Indexes are declared in model files and synchronized during seeding. The index inventory is listed in Section 4.7.2 and Appendix A.

5.6 CRUD, Transaction and Aggregation Modules

Books and members have list, get-by-ID, create, update and delete routes. Circulation routes issue/return, create/cancel reservations, renew eligible loans and read loans/queues. Issue, return and renewal use transactions. A loan can be renewed twice, only through its due date, and only when no active reservation exists for its book. Analytics routes execute six pipelines and provide overdue CSV export. Authorization and validation precede protected writes.

5.7 Application Connectivity

Browser JavaScript calls same-origin JSON endpoints. Express authenticates requests using an HTTP-only JWT cookie; route handlers call Mongoose; Mongoose connects through `MONGODB_URI` and a connection pool. Compose uses `mongodb://mongo:27017/library_management?replicaSet=rs0`; host port 27017 is exposed for Compass. JDBC/ODBC is not used; this JavaScript app uses Mongoose/MongoDB driver.

5.8 Version Control and Project Repository

Source is delivered as `Library_Management_System_Complete.zip` with app, tests, documentation and screenshots. No public GitHub/Google Drive URL is configured; the report points to the bundled archive and does not invent an external link. Upload to a faculty-approved repository if an external URL is required.

5.9 Challenges Faced and Solutions Adopted

Challenge	Solution
Coordinating loan state and stock	Transactions for issue and return
Duplicate active records	Partial unique indexes
Flexible schema consistency	Mongoose/Zod validation plus strict DB validators
Catalog and due-date access	Text, unique and compound indexes
Transaction environment	Single-node replica-set initialization in Compose
Destructive reseeding	Document reset behavior and limit seed to demo database

Chapter 6: Query Design and Performance Tuning

6.1 Representative Queries

MongoDB uses filters and aggregation stages rather than SQL statements. The matrix maps requested query types to operations in this project.

Query	MongoDB equivalent	Project example	Status
Selection	find(filter) / $match	Books by category/availability; overdue loans	Implemented
Join	$lookup	Loan rows with book and member labels	Implemented
Grouping	$group, $sum, $cond	Inventory, popularity, member activity	Implemented
Date compute	$dateDiff, $dateToString	Overdue days/fine; monthly counts	Implemented
Subquery-like	$lookup with pipeline	Constrain related documents during join	Supported, not needed by current reports
Set operation	$unionWith	Combine independent result streams	Not used in current application

Selection: available books in a category

```
db.books.find({ category: "Database", "copies.available": { $gt: 0 } })
  .sort({ title: 1 }).limit(20);
```

Join: current loans with book and member names

```
db.loans.aggregate([
  { $match: { status: "Issued" } },
  { $lookup: { from: "books", localField: "bookId", foreignField: "_id", as: "book" } },
  { $lookup: { from: "members", localField: "memberId", foreignField: "_id", as: "member" } },
  { $project: { loanId: "$_id", title: { $first: "$book.title" },
      member: { $first: "$member.name" }, dueDate: 1 } }
]);
```

Grouping: inventory by category

```
db.books.aggregate([
  { $group: { _id: "$category", titles: { $sum: 1 },
      totalCopies: { $sum: "$copies.total" },
      availableCopies: { $sum: "$copies.available" } } },
  { $set: { issuedCopies: { $subtract: ["$totalCopies", "$availableCopies"] } } },
  { $sort: { totalCopies: -1, _id: 1 } }
]);
```

Overdue report: filter, compute, join and sort

```
db.loans.aggregate([
  { $match: { status: "Issued", dueDate: { $lt: new Date() } } },
  { $set: { overdueDays: { $dateDiff: { startDate: "$dueDate",
      endDate: new Date(), unit: "day" } } } },
  { $set: { calculatedFine: { $multiply: ["$overdueDays", 10] } } },
  { $lookup: { from: "books", localField: "bookId", foreignField: "_id", as: "book" } },
  { $lookup: { from: "members", localField: "memberId", foreignField: "_id", as: "member" } },
  { $sort: { overdueDays: -1 } }
]);
```

The six report pipelines are implemented in `src/routes/analytics.js`. Aggregation stages support filtering, grouping, reshaping and computation [6]. `$lookup` supplies a join-style operation. `$unionWith` is available for set-like needs but is not used by this application; it is listed as an extension rather than claimed functionality.

6.2 Performance Tuning Strategy

Indexes align with recurring query patterns: catalog search, category browse, member overdue lookups, and FIFO reservation lookup. MongoDB notes indexes can reduce scanned documents but add storage and write overhead [5]. This application includes bounded list endpoints and an API request-size limit.

6.3 Query Plan and Benchmark Protocol

No latency or throughput benchmark is claimed. To evaluate performance later, seed the same data, warm the database consistently, run each query repeatedly, and compare median/p95 latency and explain execution statistics before/after a candidate index. Record totalKeysExamined, totalDocsExamined, nReturned, environment and dataset size.

```
db.loans.find({ memberId: "M001", status: "Issued", dueDate: { $lt: new Date() } })
  .explain("executionStats");
```

```
db.books.find({ category: "Database", "copies.available": { $gt: 0 } })
  .explain("executionStats");
```

Chapter 7: Results and Discussion

7.1 Output Screenshots and Sample Reports

Saved evidence captures the seeded dashboard, all six reports and a narrow mobile view. The dashboard shows 40 seeded titles; a separate browser CRUD verification previously showed 41 after adding B041, so counts can differ after modifying the seed state.

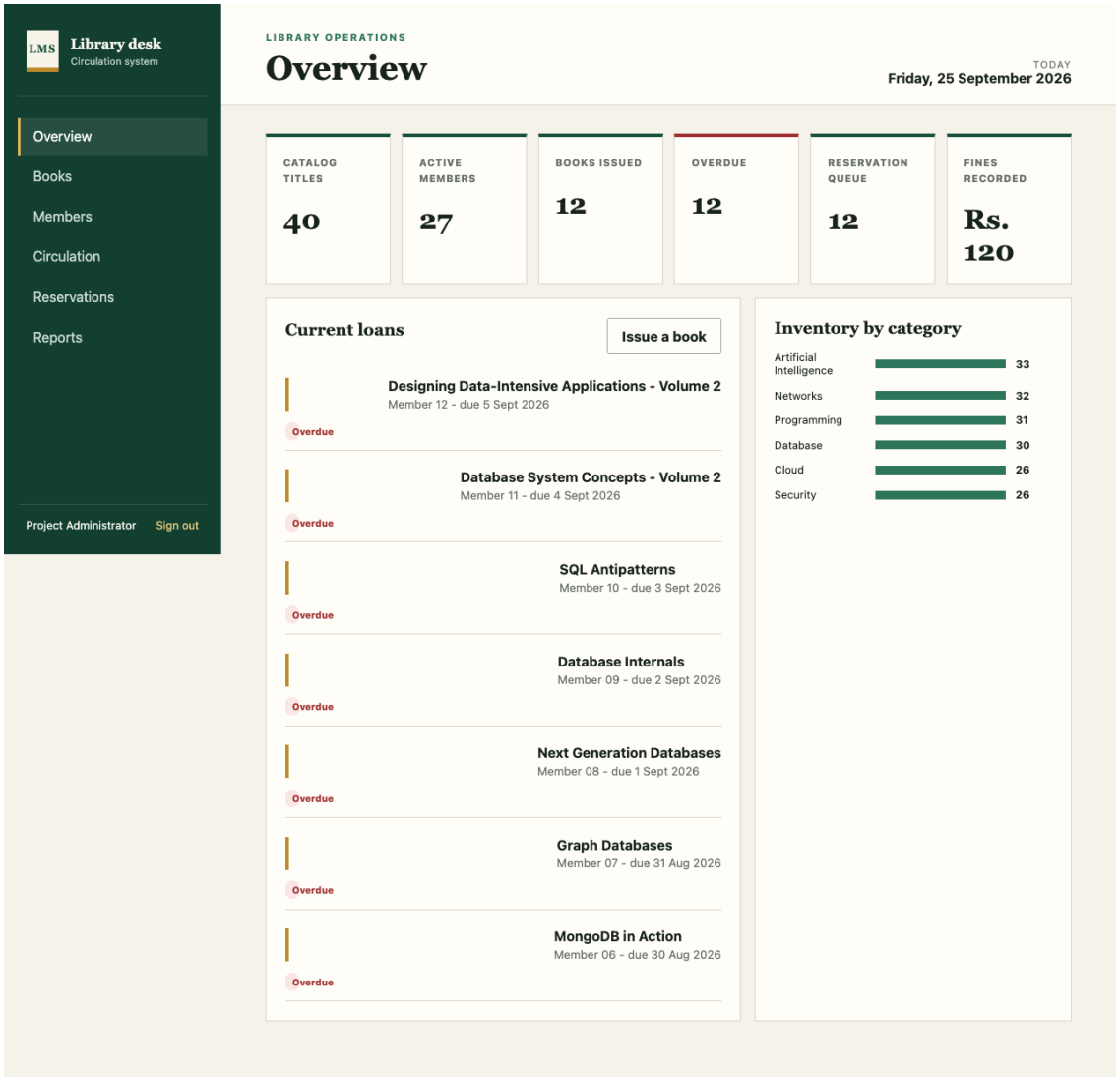


Figure 3. Seeded desktop dashboard and circulation summary.

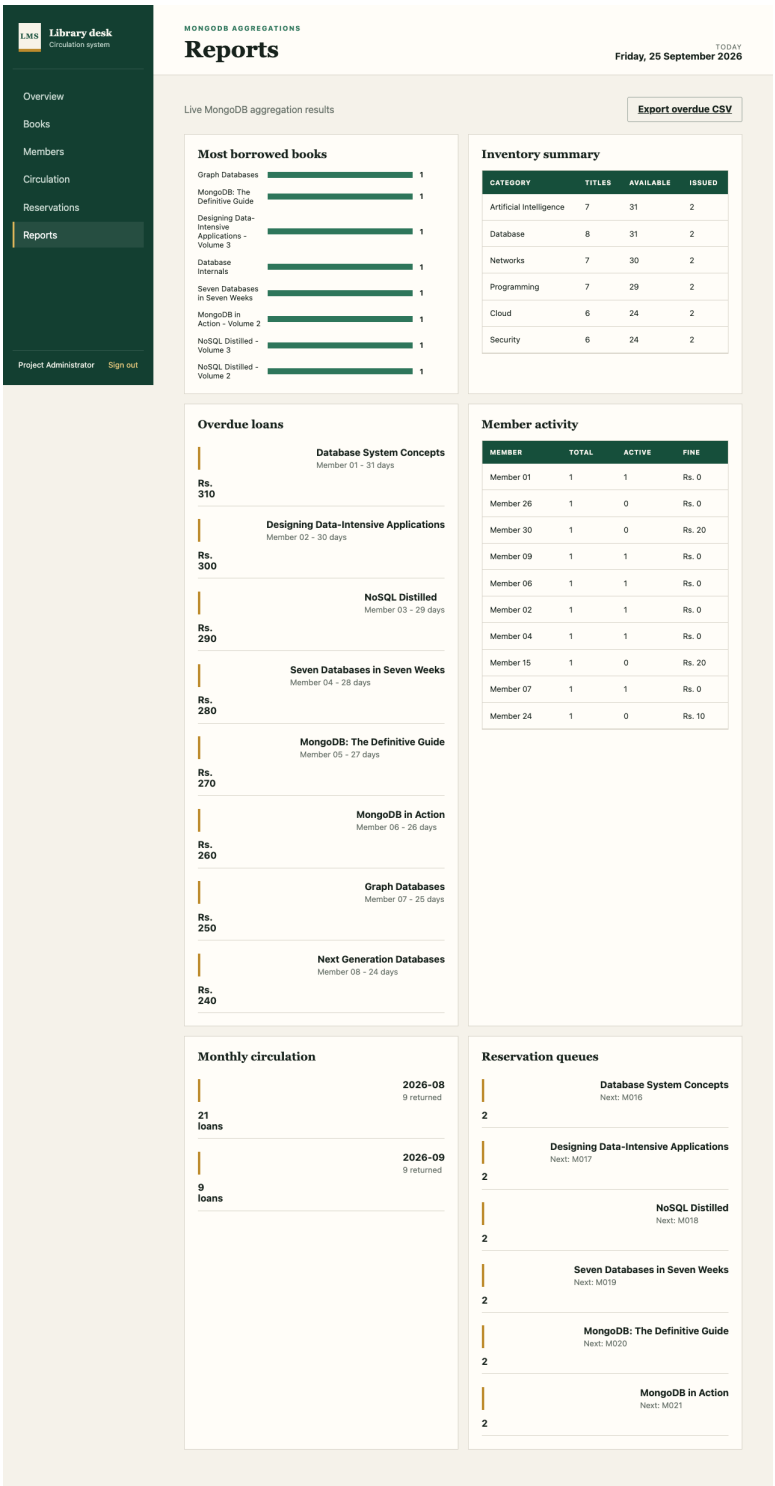


Figure 4. Six aggregation report outputs from saved application evidence.

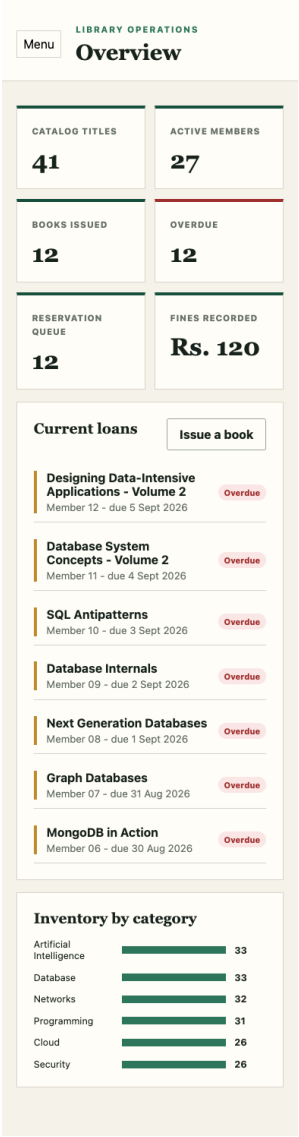


Figure 5. Responsive mobile overview.

7.2 Evaluation Against Objectives

Objective	Evidence	Evaluation
O1 model	Six collections, ER diagram, schema table and validators	Addressed for project scope
O2 sample volume	20+40+30+30+15+2 = 137	Exceeds 50-record minimum
O3 CRUD	Books/member routes and circulation interface	Implemented; demonstrate live
O4 indexes/reports	Index definitions and six pipelines	Implemented; not performance-benchmarked
O5 consistency	Issue/return transaction routes	Implemented on single-node replica set
O6 repeatability	Compose, seed, tests, manual and archive	Packaged; depends on local Docker engine

7.3 Comparison with the Existing System

Dimension	Paper/spreadsheet baseline	This project	Mature platform context
Search	Manual or split sheets	API search and indexes	Koha covers broader catalog/patron workflows [14]
Consistency	Manual stock reconciliation	Validation and transactions	Production rules need institutional configuration
Reports	Manual grouping	Six aggregation reports	Mature products provide wider reporting
Scope	Low setup, fragmented records	Small single-library demo	Not a production replacement

7.4 Discussion of Findings

A document database can still have an explicit schema. Models and validators constrain documents; references preserve separate histories; aggregation supplies summaries without exporting data. Transactions are justified because issue/return touches multiple documents. These strengths are functional design evidence, not a speed comparison or production certification.

7.5 Limitations Observed

The dataset is synthetic; no patron portal, barcode workflow or email notification exists. MongoDB does not enforce reference integrity as foreign keys. The replica set is one node. Seed is destructive to its named database. TLS, managed secrets, automated backups, recovery drills and measured performance remain production work. The classroom bcrypt configuration should be revisited for new deployments in line with current OWASP guidance [15].

Chapter 8: Conclusion and Future Work

8.1 Summary of the Work

The project specifies a staff library workflow, models six MongoDB collections, creates validators and indexes, seeds 137 documents and implements a responsive UI with protected REST endpoints. It supports catalog/member maintenance, circulation, holds, six reports and overdue CSV export.

8.2 Key Contributions

Contribution	Assessment value
End-to-end model	Connects conceptual relationships to actual collections/routes
Sample scale	137 deterministic records exceed the minimum of 50
Consistency controls	Validators, partial unique indexes and transactions map to business rules
Database analytics	Six pipelines demonstrate filtering, grouping, joins and date calculations
Reproducibility	Compose, seed, source archive, manual and evidence support demonstration

8.3 Future Enhancements

Priorities: rotate demo credentials and use TLS/secrets; consider Argon2id; deploy multiple replica-set members and rehearse backups/restores; add referential-integrity audits and an audit log; benchmark explain plans at realistic scale; add barcode scanning, notifications and patron self-service. Graph-based recommendations should be added only if relationship-path queries become a genuine requirement.

8.4 Concluding Remarks

Within its classroom scope, the project demonstrates a complete MongoDB-backed library application. Its strongest evidence is the traceable link from rules to collections, indexes, transactions, reports and visible outputs. The limitations distinguish this academic prototype from a production library deployment.

Part C: References

[1] MongoDB, Inc., 'Data Modeling in MongoDB,' MongoDB Manual. https://www.mongodb.com/docs/manual/data-modeling/. Accessed Oct. 23, 2026.

[2] MongoDB, Inc., 'Embedded Data in Your MongoDB Schema,' https://www.mongodb.com/docs/manual/data-modeling/embedding/. Accessed Oct. 23, 2026.

[3] MongoDB, Inc., 'Reference Data in Your MongoDB Schema,' MongoDB Manual v7.0. https://www.mongodb.com/docs/v7.0/data-modeling/referencing/. Accessed Oct. 23, 2026.

[4] MongoDB, Inc., 'Schema Validation,' https://www.mongodb.com/docs/manual/core/schema-validation/. Accessed Oct. 23, 2026.

[5] MongoDB, Inc., 'Indexes,' https://www.mongodb.com/docs/manual/indexes/. Accessed Oct. 23, 2026.

[6] MongoDB, Inc., 'Aggregation Operations,' https://www.mongodb.com/docs/manual/aggregation/. Accessed Oct. 23, 2026.

[7] MongoDB, Inc., 'Transactions,' https://www.mongodb.com/docs/manual/core/transactions/. Accessed Oct. 23, 2026.

[8] MongoDB, Inc., 'Replication,' https://www.mongodb.com/docs/manual/replication/. Accessed Oct. 23, 2026.

[9] MongoDB, Inc., 'Backup Methods for a Self-Managed Deployment,' https://www.mongodb.com/docs/manual/core/backups/. Accessed Oct. 23, 2026.

[10] PostgreSQL Global Development Group, 'JSON Types,' PostgreSQL 18 Documentation. https://www.postgresql.org/docs/current/datatype-json.html. Accessed Oct. 23, 2026.

[11] Apache Cassandra Project, 'Data Modeling: Introduction,' https://cassandra.apache.org/doc/stable/cassandra/developing/data-modeling/intro.html. Accessed Oct. 23, 2026.

[12] Neo4j, Inc., 'What is a Graph Database?,' https://neo4j.com/docs/getting-started/graph-database/. Accessed Oct. 23, 2026.

[13] Redis Ltd., 'Redis Data Types,' https://redis.io/docs/latest/develop/data-types/. Accessed Oct. 23, 2026.

[14] Koha Community, 'Circulation,' Koha Manual. https://koha-community.org/manual/latest/en/html/circulation.html. Accessed Oct. 23, 2026.

[15] OWASP Foundation, 'Password Storage Cheat Sheet.' https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Accessed Oct. 23, 2026.

[16] OpenJS Foundation, 'Production Best Practices: Security,' Express.js. https://expressjs.com/advanced/best-practice-security/. Accessed Oct. 23, 2026.

[17] PostgreSQL Global Development Group, 'PostgreSQL Documentation.' https://www.postgresql.org/docs/current/. Accessed Oct. 23, 2026.

[18] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill, 2019.

Appendix A: Complete Database Schema and Index Definitions

MongoDB does not use SQL CREATE TABLE DDL for this document design. The Mongoose model source below defines the collection fields and indexes; the seed script applies strict JSON Schema validators using collMod. Application checks enforce additional cross-field rules such as available <= total and referenced-record existence.

author.js

Project source: `src/models/author.js`

```
import mongoose from "mongoose";

const authorSchema = new mongoose.Schema({
  _id: { type: String, match: /^A\d{3}$/, required: true },
  name: { type: String, required: true, trim: true },
  country: { type: String, required: true, trim: true }
}, { versionKey: false });

authorSchema.index({ name: 1 });
export const Author = mongoose.model("Author", authorSchema);
```

book.js

Project source: `src/models/book.js`

```
import mongoose from "mongoose";

const bookSchema = new mongoose.Schema({
  _id: { type: String, match: /^B\d{3}$/, required: true },
  isbn: { type: String, required: true, unique: true, trim: true },
  title: { type: String, required: true, trim: true },
  authors: [{ type: String, ref: "Author", required: true }],
  category: { type: String, required: true, trim: true },
  publisher: { type: String, required: true, trim: true },
  edition: { type: Number, min: 1, default: 1 },
  year: { type: Number, min: 1900, max: 2100 },
  copies: {
    total: { type: Number, min: 0, required: true },
    available: { type: Number, min: 0, required: true }
  }
}, { timestamps: true, versionKey: false });

bookSchema.index({ title: "text", publisher: "text", isbn: "text" }, { name: "book_search" });
bookSchema.index({ category: 1, "copies.available": 1 }, { name: "category_availability" });
export const Book = mongoose.model("Book", bookSchema);
```

member.js

Project source: `src/models/member.js`

```
import mongoose from "mongoose";

const memberSchema = new mongoose.Schema({
  _id: { type: String, match: /^M\d{3}$/, required: true },
  name: { type: String, required: true, trim: true },
  department: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  phone: { type: String, required: true, trim: true },
  membershipType: { type: String, enum: ["Student", "Faculty"], required: true },
  active: { type: Boolean, default: true }
}, { timestamps: true, versionKey: false });

memberSchema.index({ name: 1, department: 1 });
export const Member = mongoose.model("Member", memberSchema);
```

loan.js

Project source: `src/models/loan.js`

```
import mongoose from "mongoose";

const loanSchema = new mongoose.Schema({
  _id: { type: String, match: /^L\d{3,}$/, required: true },
  bookId: { type: String, ref: "Book", required: true },
  memberId: { type: String, ref: "Member", required: true },
  issueDate: { type: Date, required: true },
  dueDate: { type: Date, required: true },
  returnDate: { type: Date, default: null },
  status: { type: String, enum: ["Issued", "Returned"], required: true },
  fine: { type: Number, min: 0, default: 0 },
  renewals: { type: Number, min: 0, max: 2, default: 0 },
```

```
  issuedBy: { type: String, ref: "User", required: true }
}, { timestamps: true, versionKey: false });

loanSchema.index({ memberId: 1, status: 1, dueDate: 1 }, { name: "member_active_loans" });
loanSchema.index({ bookId: 1, status: 1 }, { name: "book_loan_history" });
loanSchema.index(
  { bookId: 1, memberId: 1 },
  { unique: true, partialFilterExpression: { status: "Issued" }, name: "one_active_copy_per_member" }
);
export const Loan = mongoose.model("Loan", loanSchema);
```

reservation.js

Project source: `src/models/reservation.js`

```
import mongoose from "mongoose";

const reservationSchema = new mongoose.Schema({
  _id: { type: String, match: /^R\d{3,}$/, required: true },
  bookId: { type: String, ref: "Book", required: true },
  memberId: { type: String, ref: "Member", required: true },
  reservationDate: { type: Date, required: true },
  status: { type: String, enum: ["Active", "Fulfilled", "Cancelled"], required: true }
}, { timestamps: true, versionKey: false });

reservationSchema.index({ bookId: 1, status: 1, reservationDate: 1 }, { name: "reservation_queue" });
reservationSchema.index(
  { bookId: 1, memberId: 1 },
  { unique: true, partialFilterExpression: { status: "Active" }, name: "one_active_reservation" }
);
export const Reservation = mongoose.model("Reservation", reservationSchema);
```

user.js

Project source: `src/models/user.js`

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  passwordHash: { type: String, required: true, select: false },
  role: { type: String, enum: ["admin", "librarian"], default: "librarian" },
  active: { type: Boolean, default: true }
}, { timestamps: true, versionKey: false });

export const User = mongoose.model("User", userSchema);
```

Appendix B: Sample Data and DML Script

The complete seed script is reproduced from project source. It resets only the named demo database, hashes classroom passwords, inserts deterministic documents, synchronizes indexes, applies validators and prints counts. Run only against demonstration data.

B.1 Full seed implementation

Project source: `src/scripts/seed.js`

```
import bcrypt from "bcryptjs";
import mongoose from "mongoose";
import { connectDatabase, closeDatabase } from "../db.js";
import { Author, Book, Loan, Member, Reservation, User } from "../models/index.js";
import { addDays } from "../lib/circulation.js";

await connectDatabase();
await mongoose.connection.dropDatabase();

const passwordHash = await bcrypt.hash("admin123", 12);
const [admin, librarian] = await User.create([
  { name: "Project Administrator", email: "admin@library.edu", passwordHash, role: "admin" },
  { name: "Library Staff", email: "librarian@library.edu", passwordHash, role: "librarian" }
]);

const authorNames = [
  "Abraham Silberschatz", "Henry Korth", "S. Sudarshan", "Martin Kleppmann",
  "Pramod Sadalage", "Eric Redmond", "Kristina Chodorow", "Kyle Banker",
  "Ian Robinson", "Jim Webber", "Emil Eifrem", "Guy Harrison", "Alex Petrov",
  "Bill Karwin", "Joe Celko", "C. J. Date", "Raghu Ramakrishnan",
  "Johannes Gehrke", "Hector Garcia-Molina", "Jeffrey Ullman"
];
const countries = ["India", "USA", "UK", "Canada", "Germany"];
await Author.insertMany(authorNames.map((name, i) => ({
  _id: `A${String(i + 1).padStart(3, "0")}`,
  name,
  country: countries[i % countries.length]
})));

const titles = [
  "Database System Concepts", "Designing Data-Intensive Applications", "NoSQL Distilled",
  "Seven Databases in Seven Weeks", "MongoDB: The Definitive Guide", "MongoDB in Action",
  "Graph Databases", "Next Generation Databases", "Database Internals", "SQL Antipatterns"
];
const categories = ["Database", "Programming", "Networks", "Artificial Intelligence", "Security", "Cloud"];
const publishers = ["Pearson", "McGraw Hill", "O'Reilly", "Springer", "MIT Press"];
const books = Array.from({ length: 40 }, (_, i) => {
  const total = i < 6 ? 1 : 3 + (i % 5);
  const activeIssued = i < 12 ? 1 : 0;
  return {
    _id: `B${String(i + 1).padStart(3, "0")}`,
    isbn: `978000${String(1000000 + i).slice(-7)}`,
    title: `${titles[i % titles.length]}${i >= 10 ? ` - Volume ${Math.floor(i / 10) + 1}` : ""}`,
    authors: [`A${String((i % 20) + 1).padStart(3, "0")}`, `A${String(((i + 3) % 20) + 1).padStart(3, "0")}`],
    category: categories[i % categories.length],
    publisher: publishers[i % publishers.length],
    edition: 1 + (i % 8),
    year: 2016 + (i % 11),
    copies: { total, available: Math.max(0, total - activeIssued) }
  };
});
await Book.insertMany(books);

const departments = ["Computer Science", "Data Science", "Electronics", "Mechanical", "Business"];
await Member.insertMany(Array.from({ length: 30 }, (_, i) => ({
  _id: `M${String(i + 1).padStart(3, "0")}`,
  name: `Member ${String(i + 1).padStart(2, "0")}`,
  department: departments[i % departments.length],
  email: `member${i + 1}@vitstudent.ac.in`,
  phone: `98${String(70000000 + i).slice(-8)}`,
  membershipType: i % 6 === 0 ? "Faculty" : "Student",
  active: i % 13 !== 0
})));

const today = new Date();
today.setUTCHours(0, 0, 0, 0);
const loans = Array.from({ length: 30 }, (_, i) => {
  const active = i < 12;
  const issueDate = addDays(today, -(45 - i));
  const dueDate = addDays(issueDate, 14);
  const returnDate = active ? null : addDays(dueDate, (i % 5) - 2);
  return {
    _id: `L${String(i + 1).padStart(3, "0")}`,
    bookId: `B${String((i % 40) + 1).padStart(3, "0")}`,
```

```
    memberId: `M${String((i % 30) + 1).padStart(3, "0")}`,
    issueDate,
    dueDate,
    returnDate,
    status: active ? "Issued" : "Returned",
    fine: !active && returnDate > dueDate ? Math.ceil((returnDate - dueDate) / 86400000) * 10 : 0,
    renewals: 0,
    issuedBy: i % 2 ? librarian._id : admin._id
  };
});
await Loan.insertMany(loans);

await Reservation.insertMany(Array.from({ length: 15 }, (_, i) => ({
  _id: `R${String(i + 1).padStart(3, "0")}`,
  bookId: `B${String((i % 6) + 1).padStart(3, "0")}`,
  memberId: `M${String(((i + 15) % 30) + 1).padStart(3, "0")}`,
  reservationDate: addDays(today, -(15 - i)),
  status: i < 12 ? "Active" : "Cancelled"
})));

await Promise.all([Author.syncIndexes(), Book.syncIndexes(), Member.syncIndexes(), Loan.syncIndexes(), Reservation.syncIndexes(), User.

const validators = {
  authors: { $jsonSchema: { bsonType: "object", required: ["_id", "name", "country"] } },
  books: { $jsonSchema: { bsonType: "object", required: ["_id", "isbn", "title", "authors", "category", "copies"], properties: { copies
  members: { $jsonSchema: { bsonType: "object", required: ["_id", "name", "email", "membershipType", "active"] } },
  loans: { $jsonSchema: { bsonType: "object", required: ["_id", "bookId", "memberId", "issueDate", "dueDate", "status", "fine"], proper
  reservations: { $jsonSchema: { bsonType: "object", required: ["_id", "bookId", "memberId", "reservationDate", "status"], properties:
  users: { $jsonSchema: { bsonType: "object", required: ["name", "email", "passwordHash", "role", "active"], properties: { role: { enum
};

for (const [collection, validator] of Object.entries(validators)) {
  await mongoose.connection.db.command({ collMod: collection, validator, validationLevel: "strict", validationAction: "error" });
}

const collections = ["authors", "books", "members", "loans", "reservations", "users"];
const counts = Object.fromEntries(await Promise.all(collections.map(async (name) => [name, await mongoose.connection.db.collection(name
counts.total = Object.values(counts).reduce((sum, value) => sum + value, 0);
console.log("Seed complete", counts);
console.log("Login: admin@library.edu / admin123");
await closeDatabase();
```

Appendix C: Stored Procedures, Functions and Transactions

MongoDB server-side SQL stored procedures/triggers are not used. Equivalent business logic is implemented as Express route procedures and JavaScript helpers. Full transaction route source and fine/date functions follow.

C.1 Circulation routes

Project source: `src/routes/circulation.js`

```
import { Router } from "express";
import mongoose from "mongoose";
import { z } from "zod";
import { Book, Loan, Member, Reservation } from "../models/index.js";
import { asyncHandler } from "../lib/async-handler.js";
import { AppError, ConflictError, NotFoundError, ValidationError } from "../lib/errors.js";
import { addDays, calculateFine, renewalDecision } from "../lib/circulation.js";

export const circulationRouter = Router();
const issueInput = z.object({
  bookId: z.string().regex(/^B\d{3}$/),
  memberId: z.string().regex(/^M\d{3}$/),
  loanDays: z.coerce.number().int().min(1).max(60).default(14)
});

async function nextId(model, prefix, session) {
  const query = model.findOne({}, { _id: 1 }).sort({ _id: -1 });
  if (session) query.session(session);
  const latest = await query.lean();
  const number = latest ? Number(String(latest._id).slice(1)) + 1 : 1;
  return `${prefix}${String(number).padStart(3, "0")}`;
}

circulationRouter.get("/loans", asyncHandler(async (req, res) => {
  const filter = req.query.status ? { status: req.query.status } : {};
  const data = await Loan.aggregate([
    { $match: filter },
    { $sort: { issueDate: -1 } },
    { $limit: 100 },
    { $lookup: { from: "books", localField: "bookId", foreignField: "_id", as: "book" } },
    { $lookup: { from: "members", localField: "memberId", foreignField: "_id", as: "member" } },
    { $set: { book: { $first: "$book" }, member: { $first: "$member" } } },
    { $project: { _id: 1, bookId: 1, memberId: 1, title: "$book.title", memberName: "$member.name", issueDate: 1, dueDate: 1, returnDat
  ]);
  res.json({ status: "success", data });
}));

circulationRouter.post("/issue", asyncHandler(async (req, res) => {
  const input = issueInput.parse(req.body);
  const session = await mongoose.startSession();
  let created;
  try {
    await session.withTransaction(async () => {
      const [book, member, duplicate] = await Promise.all([
        Book.findById(input.bookId).session(session),
        Member.findById(input.memberId).session(session),
        Loan.findOne({ bookId: input.bookId, memberId: input.memberId, status: "Issued" }).session(session)
      ]);
      if (!book) throw new NotFoundError("Book not found");
      if (!member) throw new NotFoundError("Member not found");
      if (!member.active) throw new ValidationError("Member account is inactive");
      if (duplicate) throw new ConflictError("This member already has an active loan for this book");
      if (book.copies.available < 1) throw new ConflictError("No copy is currently available");

      const issueDate = new Date();
      const loanId = await nextId(Loan, "L", session);
      [created] = await Loan.create([{
        _id: loanId,
        bookId: book._id,
        memberId: member._id,
        issueDate,
        dueDate: addDays(issueDate, input.loanDays),
        returnDate: null,
        status: "Issued",
        fine: 0,
        issuedBy: req.user.sub
      }], { session });
      await Book.updateOne({ _id: book._id, "copies.available": { $gt: 0 } }, { $inc: { "copies.available": -1 } }, { session });
    });
  } catch (error) {
    if (error?.errorLabels?.includes("TransientTransactionError")) {
      throw new AppError("Transaction conflict. Please retry the issue operation", 409);
    }
```

```
    throw error;
  } finally {
    await session.endSession();
  }
  res.status(201).json({ status: "success", data: created });
}));

circulationRouter.post("/return/:loanId", asyncHandler(async (req, res) => {
  const session = await mongoose.startSession();
  let result;
  try {
    await session.withTransaction(async () => {
      const loan = await Loan.findOne({ _id: req.params.loanId, status: "Issued" }).session(session);
      if (!loan) throw new NotFoundError("Active loan not found");
      const returnedAt = new Date();
      const { overdueDays, fine } = calculateFine(loan.dueDate, returnedAt);

      await Loan.updateOne(
        { _id: loan._id, status: "Issued" },
        { $set: { status: "Returned", returnDate: returnedAt, fine } },
        { session }
      );
      await Book.updateOne(
        { _id: loan.bookId, $expr: { $lt: ["$copies.available", "$copies.total"] } },
        { $inc: { "copies.available": 1 } },
        { session }
      );
      const nextReservation = await Reservation.findOne({ bookId: loan.bookId, status: "Active" })
        .sort({ reservationDate: 1 })
        .session(session);
      if (nextReservation) {
        await Reservation.updateOne({ _id: nextReservation._id }, { $set: { status: "Fulfilled" } }, { session });
      }
      result = { loanId: loan._id, returnedAt, overdueDays, fine, fulfilledReservation: nextReservation?._id || null };
    });
  } finally {
    await session.endSession();
  }
  res.json({ status: "success", data: result });
}));

circulationRouter.post("/renew/:loanId", asyncHandler(async (req, res) => {
  const session = await mongoose.startSession();
  let result;
  try {
    await session.withTransaction(async () => {
      const loan = await Loan.findById(req.params.loanId).session(session);
      if (!loan) throw new NotFoundError("Loan not found");
      const hasActiveReservation = Boolean(await Reservation.exists({ bookId: loan.bookId, status: "Active" }).session(session));
      const decision = renewalDecision(loan, hasActiveReservation);
      if (!decision.allowed) throw new ValidationError(decision.reason);
      loan.dueDate = addDays(loan.dueDate, 14);
      loan.renewals = (loan.renewals || 0) + 1;
      await loan.save({ session });
      result = { loanId: loan._id, dueDate: loan.dueDate, renewals: loan.renewals, maximumRenewals: 2 };
    });
  } finally {
    await session.endSession();
  }
  res.json({ status: "success", data: result });
}));

circulationRouter.get("/reservations", asyncHandler(async (_req, res) => {
  const data = await Reservation.aggregate([
    { $sort: { reservationDate: 1 } },
    { $lookup: { from: "books", localField: "bookId", foreignField: "_id", as: "book" } },
    { $lookup: { from: "members", localField: "memberId", foreignField: "_id", as: "member" } },
    { $project: { _id: 1, bookId: 1, memberId: 1, title: { $first: "$book.title" }, memberName: { $first: "$member.name" }, reservation
  ]);
  res.json({ status: "success", data });
}));

circulationRouter.post("/reserve", asyncHandler(async (req, res) => {
  const input = issueInput.pick({ bookId: true, memberId: true }).parse(req.body);
  const [book, member] = await Promise.all([Book.findById(input.bookId), Member.findById(input.memberId)]);
  if (!book) throw new NotFoundError("Book not found");
  if (!member || !member.active) throw new ValidationError("An active member is required");
  if (book.copies.available > 0) throw new ValidationError("This book is available and should be issued instead");
  const id = await nextId(Reservation, "R");
  const reservation = await Reservation.create({
    _id: id,
    bookId: input.bookId,
    memberId: input.memberId,
    reservationDate: new Date(),
    status: "Active"
  });
  res.status(201).json({ status: "success", data: reservation });
}));
```

```
circulationRouter.patch("/reservations/:id/cancel", asyncHandler(async (req, res) => {
  const reservation = await Reservation.findOneAndUpdate(
    { _id: req.params.id, status: "Active" },
    { $set: { status: "Cancelled" } },
    { new: true }
  );
  if (!reservation) throw new NotFoundError("Active reservation not found");
  res.json({ status: "success", data: reservation });
}));
```

C.2 Date and fine functions

Project source: `src/lib/circulation.js`

```
export function calculateFine(dueDate, returnedAt = new Date(), dailyRate = 10) {
  const due = new Date(dueDate);
  const returned = new Date(returnedAt);
  const overdueMs = returned.setUTCHours(0, 0, 0, 0) - due.setUTCHours(0, 0, 0, 0);
  const overdueDays = Math.max(0, Math.floor(overdueMs / 86400000));
  return { overdueDays, fine: overdueDays * dailyRate };
}

export function addDays(date, days) {
  return new Date(new Date(date).getTime() + days * 86400000);
}

export function startOfUtcDay(date = new Date()) {
  const value = new Date(date);
  value.setUTCHours(0, 0, 0, 0);
  return value;
}

export function isOverdue(dueDate, today = new Date()) {
  return startOfUtcDay(dueDate) < startOfUtcDay(today);
}

export function renewalDecision(loan, hasActiveReservation, today = new Date(), maximumRenewals = 2) {
  if (loan.status !== "Issued") return { allowed: false, reason: "Only active loans can be renewed" };
  if ((loan.renewals || 0) >= maximumRenewals) return { allowed: false, reason: "Maximum renewals reached" };
  if (isOverdue(loan.dueDate, today)) return { allowed: false, reason: "Overdue loans cannot be renewed" };
  if (hasActiveReservation) return { allowed: false, reason: "Another member is waiting for this book" };
  return { allowed: true, reason: null };
}
```

Appendix D: Complete Query Set with Outputs

The analytics source below contains dashboard counts, all six report pipelines and overdue CSV export. Appendix F contains screenshots of report results. Values depend on seeded data and request date.

D.1 Analytics route source

Project source: `src/routes/analytics.js`

```
import { Router } from "express";
import { Book, Loan, Member, Reservation } from "../models/index.js";
import { asyncHandler } from "../lib/async-handler.js";

export const analyticsRouter = Router();

analyticsRouter.get("/dashboard", asyncHandler(async (_req, res) => {
  const today = new Date();
  today.setUTCHours(0, 0, 0, 0);
  const [books, members, activeLoans, overdue, activeReservations, fines] = await Promise.all([
    Book.countDocuments(),
    Member.countDocuments({ active: true }),
    Loan.countDocuments({ status: "Issued" }),
    Loan.countDocuments({ status: "Issued", dueDate: { $lt: today } }),
    Reservation.countDocuments({ status: "Active" }),
    Loan.aggregate([{ $group: { _id: null, total: { $sum: "$fine" } } }])
  ]);
  res.json({
    status: "success",
    data: { books, members, activeLoans, overdue, activeReservations, fines: fines[0]?.total || 0 }
  });
}));

analyticsRouter.get("/reports", asyncHandler(async (_req, res) => {
  const today = new Date();
  today.setUTCHours(0, 0, 0, 0);
  const [inventory, popularBooks, overdueLoans, memberActivity, monthlyCirculation, reservationQueues] = await Promise.all([
    Book.aggregate([
      { $group: { _id: "$category", titles: { $sum: 1 }, totalCopies: { $sum: "$copies.total" }, availableCopies: { $sum: "$copies.avai
      { $set: { issuedCopies: { $subtract: ["$totalCopies", "$availableCopies"] } } },
      { $sort: { totalCopies: -1, _id: 1 } }
    ]),
    Loan.aggregate([
      { $group: { _id: "$bookId", borrowCount: { $sum: 1 } } },
      { $sort: { borrowCount: -1 } },
      { $limit: 8 },
      { $lookup: { from: "books", localField: "_id", foreignField: "_id", as: "book" } },
      { $project: { _id: 0, bookId: "$_id", title: { $first: "$book.title" }, borrowCount: 1 } }
    ]),
    Loan.aggregate([
      { $match: { status: "Issued", dueDate: { $lt: today } } },
      { $set: { overdueDays: { $dateDiff: { startDate: "$dueDate", endDate: today, unit: "day" } } } },
      { $set: { calculatedFine: { $multiply: ["$overdueDays", 10] } } },
      { $lookup: { from: "books", localField: "bookId", foreignField: "_id", as: "book" } },
      { $lookup: { from: "members", localField: "memberId", foreignField: "_id", as: "member" } },
      { $project: { _id: 1, bookId: 1, memberId: 1, title: { $first: "$book.title" }, memberName: { $first: "$member.name" }, dueDate:
      { $sort: { overdueDays: -1 } }
    ]),
    Loan.aggregate([
      { $group: { _id: "$memberId", totalLoans: { $sum: 1 }, activeLoans: { $sum: { $cond: [{ $eq: ["$status", "Issued"] }, 1, 0] } },
      { $sort: { totalLoans: -1 } },
      { $limit: 10 },
      { $lookup: { from: "members", localField: "_id", foreignField: "_id", as: "member" } },
      { $project: { _id: 0, memberId: "$_id", name: { $first: "$member.name" }, totalLoans: 1, activeLoans: 1, totalFine: 1 } }
    ]),
    Loan.aggregate([
      { $group: { _id: { $dateToString: { format: "%Y-%m", date: "$issueDate" } }, loans: { $sum: 1 }, returns: { $sum: { $cond: [{ $eq
      { $sort: { _id: 1 } },
      { $limit: 12 }
    ]),
    Reservation.aggregate([
      { $match: { status: "Active" } },
      { $sort: { reservationDate: 1 } },
      { $group: { _id: "$bookId", queueLength: { $sum: 1 }, nextMember: { $first: "$memberId" }, oldestRequest: { $first: "$reservation
      { $lookup: { from: "books", localField: "_id", foreignField: "_id", as: "book" } },
      { $project: { _id: 0, bookId: "$_id", title: { $first: "$book.title" }, queueLength: 1, nextMember: 1, oldestRequest: 1 } },
      { $sort: { queueLength: -1, oldestRequest: 1 } }
    ])
  ]);
  res.json({ status: "success", data: { inventory, popularBooks, overdueLoans, memberActivity, monthlyCirculation, reservationQueues }
}));

analyticsRouter.get("/export/overdue.csv", asyncHandler(async (_req, res) => {
  const today = new Date();
  today.setUTCHours(0, 0, 0, 0);
  const rows = await Loan.find({ status: "Issued", dueDate: { $lt: today } }).sort({ dueDate: 1 }).lean();
```

```
  const csv = ["loan_id,book_id,member_id,due_date", ...rows.map((r) => `${r._id},${r.bookId},${r.memberId},${r.dueDate.toISOString()}`
  res.type("text/csv").attachment("overdue-loans.csv").send(csv);
}));
```

D.2 API query examples

```
GET /api/books?category=Database&available=true
GET /api/books?search=MongoDB&limit=20
GET /api/members?search=Data%20Science
GET /api/circulation/loans?status=Issued
POST /api/circulation/renew/L001
GET /api/analytics/dashboard
GET /api/analytics/reports
GET /api/analytics/export/overdue.csv
```

D.3 Seed-state output evidence

Query/report	Saved evidence
Counts	20 authors, 40 books, 30 members, 30 loans, 15 reservations, 2 users = 137
Dashboard	40 titles; 27 active members; 12 active loans; 12 active reservations in saved seeded-state capture
Inventory/popular titles	Six categories and top borrowed titles shown in reports capture
Overdue/member reports	Member names, due days, calculated fines and activity tables shown
Monthly/queue reports	Monthly circulation and reservation queues shown

Screenshots were captured 25 September 2026. Overdue results change with the current date because overdue days are computed when requested.

Appendix E: Full-Size ER Diagram and Logical Schema

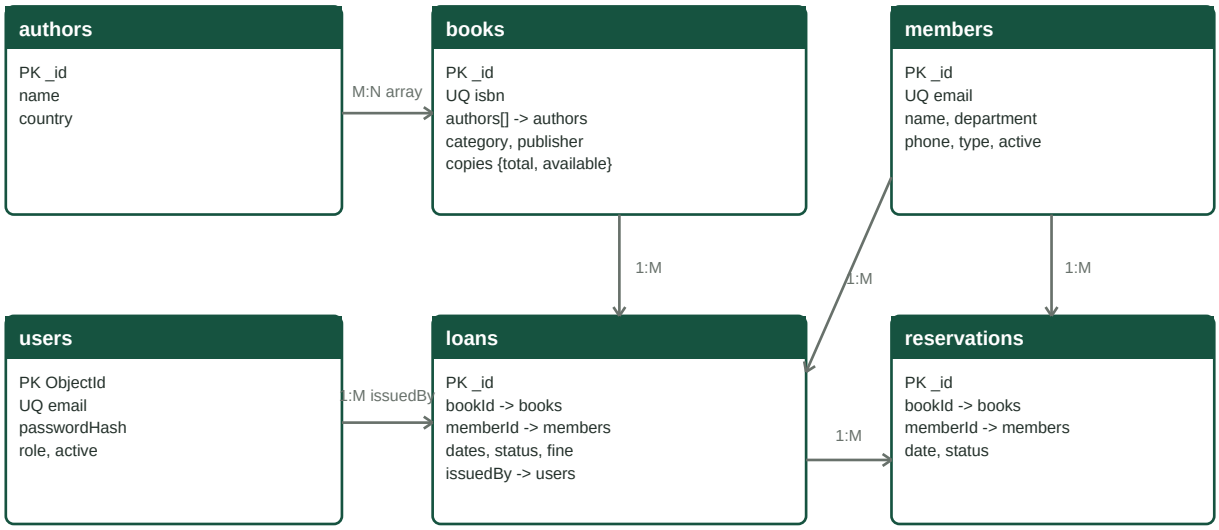


Figure E1. Full-size conceptual ER diagram; references are application-level.

Collection	Primary key	Unique key	Reference fields
authors	_id A###	-	Referenced from books.authors[]
books	_id B###	isbn	authors[] -> authors
members	_id M###	email	Referenced from loans/reservations
loans	_id L###	Partial active pair	bookId, memberId, issuedBy
reservations	_id R###	Partial active pair	bookId, memberId
users	_id ObjectId	email	loans.issuedBy

Detailed logical field types, sizes, constraints and meanings are listed in Chapter 4, Section 4.5; full model definitions are in Appendix A.

Appendix F: Application Screenshots

These screenshots are saved project evidence under docs/evidence and show the application interface, not mockups.

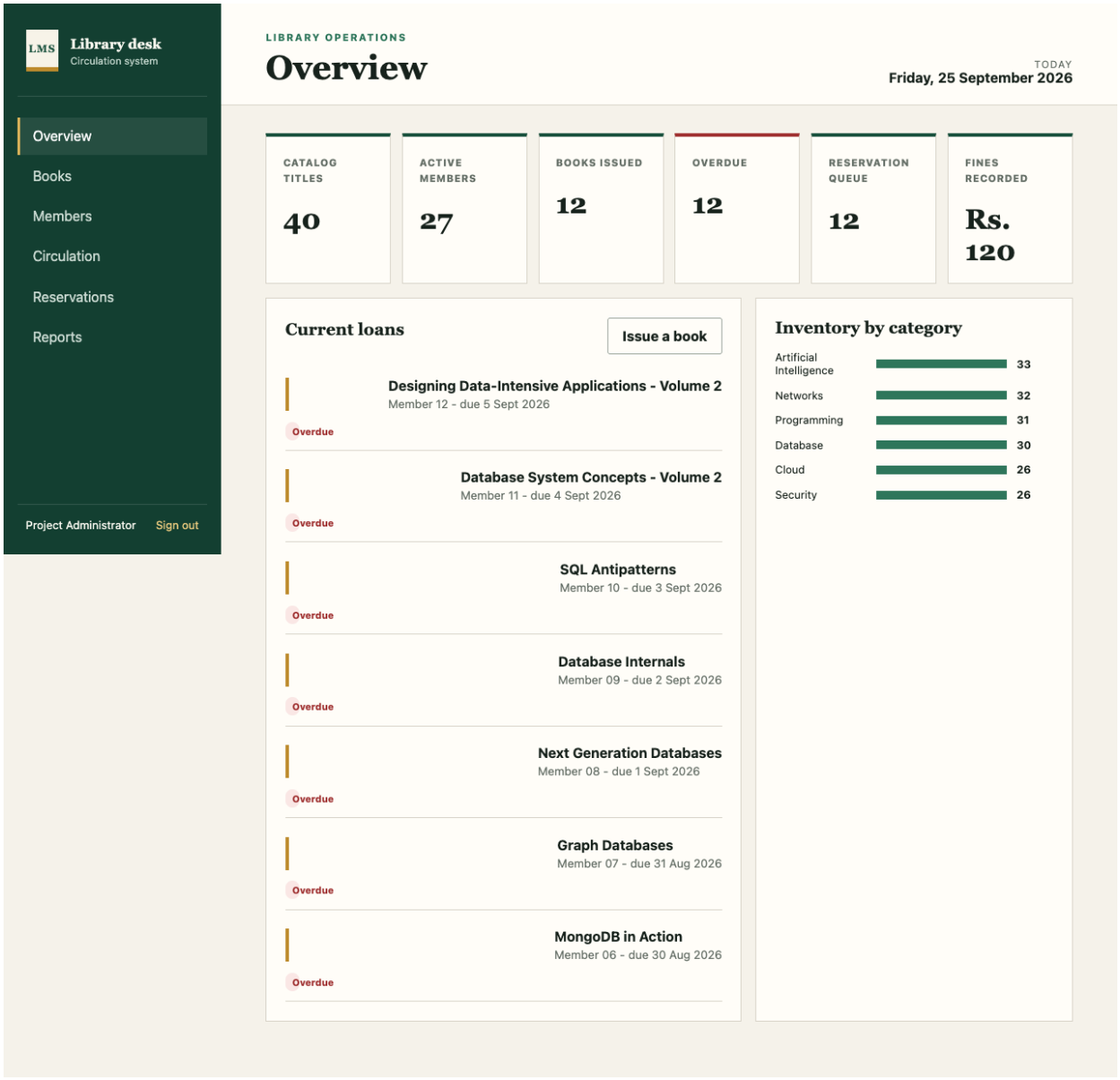


Figure F1. Desktop overview.

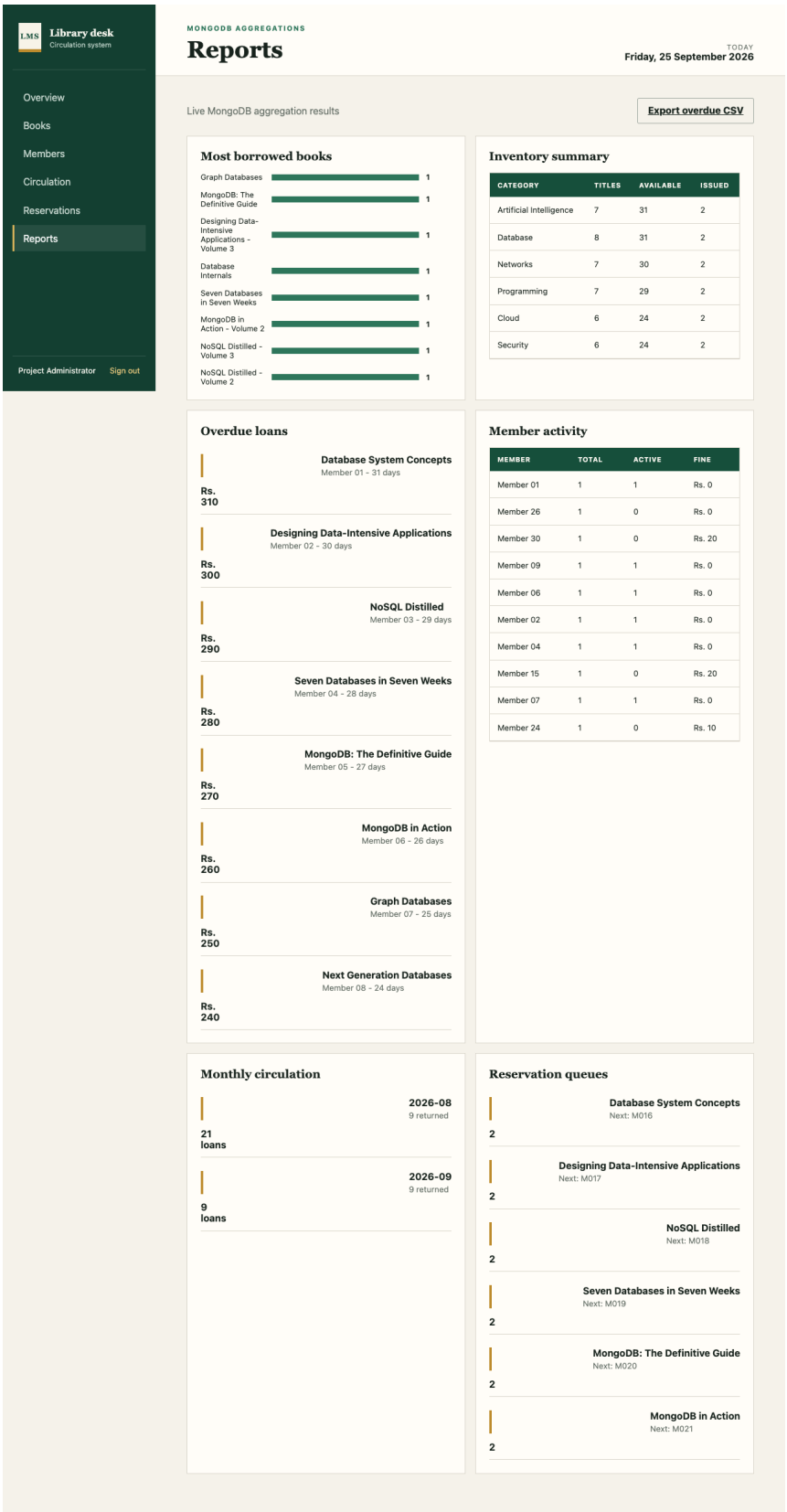


Figure F2. Six aggregation report sections.

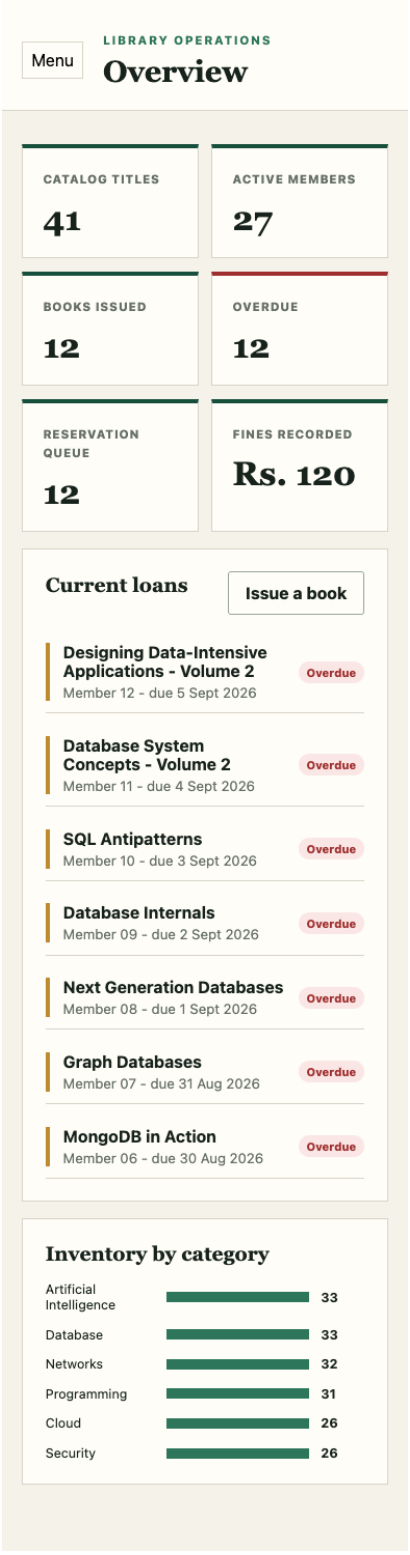


Figure F3. Responsive mobile overview.

Appendix G: User Manual

G.1 Start the application

```
# Open Docker Desktop and wait for Engine running
cd Library_Management_System_Complete
docker compose up -d --build
docker compose exec app npm run seed
docker compose ps
open http://localhost:3000
```

G.2 Demonstration accounts

Administrator: admin@library.edu / admin123. Librarian: librarian@library.edu / admin123. Classroom credentials only; change before any deployment.

G.3 Core workflows

Task	Steps
Catalog	Open Books; search title/ISBN/publisher; Add/Edit. Deletion is restricted to admin and zero-stock records.
Members	Open Members; register/edit. Active loans block member deletion.
Issue	Choose Issue a book, available title and active member. Loan and stock decrement commit together.
Renew	Choose Renew on an active loan. Adds 14 days, at most twice, only through due date and if no active hold exists; update is transactional.
Return	Choose Return. Fine is computed, loan closes, stock restores and oldest hold is fulfilled transactionally.
Reserve	Create a hold only if no copy is available; queue is ordered by reservation time.
Reports	Open Reports; inspect six pipelines and export overdue CSV.
Compass	Connect mongodb://localhost:27017/?replicaSet=rs0; inspect library_management collections, counts, indexes and validators.

G.4 Manual backup and restore

```
# Create an archive from the MongoDB container
docker compose exec -T mongo mongodump --db library_management --archive > library_management.archive

# Restore (replaces same-name collections in the demo database)
docker compose exec -T mongo mongorestore --drop --archive < library_management.archive
```

Store the archive outside the project and protect it because it contains member/staff records. Production requires secured off-device backups and restore tests [9].

G.5 Troubleshooting

Symptom	Action
Docker says manually paused	Resume Docker Desktop and wait for Engine running; confirm `docker info` has Server information.
App cannot connect	Check `docker compose ps` and Mongo health; inspect logs.
Transaction unsupported	Keep MongoDB configured as the Compose replica set.
No/changed records	Rerun seed only to reset the demo database; it drops that database.
Port conflict	Stop the process or change Compose port mappings.

G.6 Verification commands

```
npm run check
npm test
npm run test:api  # requires running app and database
npm run seed      # destructive demo-data reset
```

Source archive: Library_Management_System_Complete.zip accompanies this report and contains README, API notes, demo script, rubric checklist, test code and evidence screenshots.